FrontLine: Capstone Checkpoint (Tree of Thought)

Is ToT a good fit?

Yes. Each turn, the Player Agent must pick 1 move from about 10 legal moves (between 3 and 19 in my games). A move that looks good can lose badly after the enemy replies. ToT lets the agent look at a few options and check each one before choosing.

ToT is already built and runs in every game.

Where ToT is used

ToT runs in the **decide** step of each turn. The turn loop is:

```
observe → retrieve lessons → decide → validate → apply
```

Why a single chain of thought fails here:

- **It commits too early.** The LLM tends to take the first move that sounds good, without comparing it to others.
- **It is bad at the math.** Attack power and supply are easy to get wrong. Before ToT, 12.5% of the LLM's moves were illegal. With ToT, it is 1% to 3%. Other changes were made at the same time, so this is not a clean test.
- **It can't see the reply.** A move can look strong until the enemy answers it. Checking that needs a lookahead the LLM can't do reliably in its head.

How the tree is built

Part	In FrontLine
Thought	One candidate move plus a one-sentence reason
Root	The current board
Node	A board after a move
Branch	One of the 3 candidate moves
Depth	2 levels: my move, then the enemy's best reply
Branching	3 moves from the LLM, then every enemy reply (about 10), checked by the game engine

End	After all 3 branches are scored, the LLM picks 1. The agent then plays that move.

The LLM only writes the first level. The game engine handles the enemy-reply level, because it can check every reply quickly and exactly.

Search and scoring

Search strategy: beam search, width 3, one step deep.

The LLM suggests 3 different moves, such as expand, defend, and attack, and all 3 are kept. This fits because:

- 3 options are enough to avoid committing too early.
- Each LLM call is slow (a turn takes about 40 seconds), so a wider or deeper tree costs too much time.
- The engine already checks the enemy's reply, so the LLM does not need to go deeper.

How each branch is scored:

- **Who scores:** the game engine (`score_2ply`), not the LLM.
- **Score:** 10 × (my cells − enemy cells) + (my strength − enemy strength).
- **Lookahead:** the engine plays the move and every possible enemy reply. It keeps the reply that is worst for me, since that is what the greedy bot would play.
- **The LLM sees:** each move's score now, its score after the enemy's best reply, and that reply.

Pruning and failures:

- **Illegal candidates:** these are marked ILLEGAL and not scored. This happened to 91 of 717 candidates (13%).
- **Bad pick answer:** if the LLM's answer can't be read, the agent takes the move with the best 2-ply score.
- **Bad candidate list:** if the list can't be read, the turn counts as an illegal move. The game continues.

Ties:

- **How often:** 120 of 239 turns had a tie for the best score.
- **Who breaks them:** the LLM picks among the tied moves, using the board and any lessons.

Cost limits:

Step	Limit
Tool calls before proposing	Up to 4 rounds
Proposing and picking	1 LLM call each
	2,000 tokens at most, so a runaway

Each LLM answer	answer can’t hang the game
Candidate scoring	Plain Python, takes well under a second

Roles and tools

Role	What does it	Tool	Why
Thought generator	The LLM suggests 3 moves	LangChain `ChatOllama` (qwen3:32b) with a Pydantic schema	Structured output gives clean moves, and LangChain handles tool calls
Evaluator	The game engine scores each move 2 levels deep	Plain Python (`tools.score_2ply`)	Exact and fast, with no LLM guesswork
Controller	Runs each turn, then the LLM picks the move	LangGraph match loop	Clear steps, and each step is traced
Memory / state	The game state, the 3 candidates, and their scores	LangGraph state, saved to the trace	Every branch can be reviewed after the game

CrewAI: used for the review crew after each game (analyst, critic, curator), not for ToT. A crew would add several LLM calls to every turn.

MCP: not used. Everything runs in one Python process, and LangGraph already passes the state between steps, so MCP would add no benefit.

Risk and mitigation

Risk: the LLM ignores the evaluator.

- **What happens:** the LLM picks a move with a lower score, often because a lesson pushed it that way.
- **How often:** the LLM picked the best-scored move on 179 of 239 turns (75%).
- **Example:** in game on2-g01, turn 14, it chose a move scoring -56 over one scoring -23.

Mitigation (next step): add a guard in the pick step. If the chosen move scores far below the best one, play the best one instead. I will test this by comparing win rates with and without the guard.